



Recover

Problem set



برای این مسئله، ما چند روز گذشته را به عکس گرفتن از دور و اطراف محوطه دانشگاه گذرانیدیم و آنها را در کارت حافظه یک دوربین دیجیتالی با فرمت JPEG ذخیره کردیم. متأسفانه ما همه آنها را پاک کردیم! خوشبختانه پاک شدن در دنیای کامپیوتر به معنی فراموش شدن نیست. با اینکه دوربین اصرار بر خالی بودن کارت حافظه می‌کند اما کاملاً مطمئنیم که این کاملاً درست نیست. ما امیدواریم (انتظار داریم!) که شما برنامه‌ای بنویسید که این عکس‌ها را برای ما بازیابی کند!

در فایلی به اسم `recover.c` در فولدري به اسم `recover` ، برنامه‌ای بنویسید که فایل‌های JPEG را از یک کارت حافظه بازیابی کند.

برای این مسئله، شما باید عملکرد کدی که توسط تیم CS50 در اختیار شما قرار گرفته را ارتقا دهید.

Distribution code را دانلود کنید.

در <https://cs50.dev> وارد شوید، روی ترمینال کلیک کرده و `cd` را به تنهایی اجرا کنید، ترمینال شما باید شبیه نمونه زیر باشد:

```
$
```

حالا دستور زیر را اجرا کنید تا فایل ZIP به اسم `recover.zip` را دانلود کنید:

```
wget https://cdn.cs50.net/2025/x/psets/4/recover.zip
```

حالا با اجرا کردن دستور زیر فولد ری به اسم `recover` بسازید:

```
unzip recover.zip
```

از حالا به بعد دیگر نیازی به فایل zip ندارید پس با دستور زیر به همراه Y آن را حذف کنید:

```
rm recover.zip
```

حالا دستور زیر را اجرا کنید تا به فولدر مورد نظر منتقل شوید:

```
cd recover
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
recover/ $
```

حال اگر دستور `ls` را اجرا کنید می‌توانید فایل‌های `recover.c` و `card.raw` را مشاهده کنید.

با اینکه فایل‌های JPEG از فایل‌های BMP پیچیده‌تر هستند، اما فایل‌های JPEG هم امضا (signature) دارند که الگویی از بایت‌ها هستند که این فایل‌ها را از بقیه انواع فایل متمایز می‌کنند. اولین سه بایت فایل‌های JPEG به صورت زیر است:

```
0xff 0xd8 0xff
```

این‌ها را باید از چپ به راست بخوانید. بایت چهارم می‌تواند: 0xe0، 0xe1، 0xe2، 0xe3، 0xe4، 0xe5، 0xe6، 0xe7، 0xe8، 0xe9، 0xea، 0xeb، 0xec، 0xed، 0xee و یا 0xef باشد. به عبارت دیگر، چهار بیت اول از بایت چهارم برابر با 1110 هستند.

اگر در جاهایی که برای محل ذخیره عکس است، این الگو را پیدا کردید، بدین معناست که این فایل‌ها JPEG هستند. البته ممکن است این الگوها را بطور اتفاقی بر روی یک دیسک پیدا کنید؛ پس بازیابی داده به این روش، یک روش قطعی نیست.

خوشبختانه دوربین‌های دیجیتال عکس‌ها را به صورت پیوسته روی کارت حافظه ذخیره می‌کنند. این عکس‌ها بلافاصله بعد از گرفته شدن ذخیره می‌شوند. به همین ترتیب، شروع یک فایل JPEG معمولاً نشانه‌ی پایان فایل دیگری است. با این حال، دوربین‌های دیجیتال اغلب کارت‌ها را با یک سیستم فایل FAT مقداردهی اولیه می‌کنند که "اندازه بلوک" آن 512 بایت (B) است. این بدان معناست که این دوربین‌ها فقط در واحدهای ۵۱۲ بایتی روی این کارت‌ها می‌نویسند. بنابراین، یک عکس ۱ مگابایتی (یعنی ۱'۰۴۸'۵۷۶ بایت) روی یک کارت حافظه ۱۰۴۸۵۷۶ ÷ ۵۱۲ = ۲۰۴۸ "بلوک" را اشغال می‌کند. اما یک عکس که مثلاً فقط یک بایت کوچک‌تر باشد (یعنی ۱'۰۴۸'۵۷۵ بایت) نیز همین مقدار فضا را اشغال می‌کند! فضای هدر رفته روی دیسک "فضای slack" نامیده می‌شود. محققان جرم‌شناسی دیجیتال اغلب فضای slack را برای یافتن بقایای داده‌های مشکوک بررسی می‌کنند.

نتیجه‌ی همه‌ی این جزئیات این است که شما، به عنوان یک محقق، احتمالاً می‌توانید برنامه‌ای بنویسید که روی یک نسخه‌ی کپی از کارت حافظه‌ی من حرکت کرده و به دنبال امضای فایل‌های JPEG بگردد. هر بار که یک امضا پیدا کردید، می‌توانید یک فایل جدید برای نوشتن باز کرده و شروع به پر کردن آن فایل با بایت‌های موجود در کارت حافظه‌ی من کنید و فقط زمانی آن فایل را ببندید که به امضای دیگری برخورد کنید.

علاوه بر این، به جای خواندن بایت‌های کارت حافظه‌ی من به‌صورت تک‌به‌تک، می‌توانید برای افزایش کارایی، ۵۱۲ بایت را به‌طور همزمان در یک حافظه‌ی موقت بخوانید. به لطف سیستم فایل FAT، می‌توانید اطمینان داشته باشید که امضاهاى JPEG در "مرز بلوک‌ها" قرار دارند. به این معنا که فقط کافی است این امضاها را در چهار بایت اول هر بلوک جستجو کنید.

البته در نظر داشته باشید که فایل‌های JPEG می‌توانند در بلوک‌های متوالی گسترش یابند که در غیر این صورت، هیچ فایل JPEG نمی‌توانست بزرگتر از ۵۱۲ بایت باشد. اما ممکن است آخرین بایت یک فایل JPEG دقیقاً در انتهای یک بلوک قرار نگیرد. به یاد داشته باشید که امکان وجود فضای بلااستفاده (slack space) وجود دارد. اما نگران نباشید. چون این کارت حافظه هنگام شروع عکاسی کاملاً نو بوده، به احتمال زیاد توسط سازنده با مقدار 0 پر شده است (اصطلاحاً "zeroed" شده است). بنابراین، هر فضای بلااستفاده‌ای که باقی بماند نیز از 0ها پر خواهد بود. اگر این صفرهای اضافی در فایل‌های JPEG بازیابی‌شده شما قرار بگیرند، مشکلی ایجاد نخواهد کرد و تصاویر همچنان قابل مشاهده خواهند بود.

حالا، من فقط یک کارت حافظه دارم، اما تعداد شما زیاد است! بنابراین، من یک گام پیش رفتم و یک "forensic image" از این کارت تهیه کرده‌ام و محتویات آن را بایت به بایت در فایل به نام `card.raw` ذخیره کرده‌ام. برای اینکه وقت شما را با جستجو در میان میلیون‌ها صفر هدر ندهم، فقط چند مگابایت اول کارت حافظه را تصویربرداری کرده‌ام. اما در نهایت باید متوجه باشید که این رونوشت شامل ۵۰ فایل JPEG است.

برنامه‌ای به اسم `recover` بنویسید که فایل‌های JPEG را از یک نسخه رونوشت قانونی بازیابی کند.

- برنامه خود را در فایلی به اسم `recover.c` در فولدري به اسم `recover` پیاده‌سازی کنید.
- برنامه شما باید فقط یک آرگومان خط فرمان بپذیرد که همان اسم نسخه رونوشت قانونی‌ای است که فایل‌های JPEG باید از آن بازیابی شوند.
- اگر برنامه شما دقیقاً با تنها یک آرگومان اجرا نشد، باید به کاربر نحوه استفاده صحیح را یادآوری کند و 1 را برگرداند.
- اگر نسخه رونوشت قانونی برای خواندن باز نشد، باید به کاربر اطلاع داده شود و Main باید 1 را برگرداند.
- فایل‌هایی که می‌سازید باید با اسم `###.jpg`، که `###` یک عدد سه رقمی است، ذخیره شوند. این فایل‌ها باید از 000 شروع شوند.
- اگر برنامه شما از `malloc` استفاده می‌کند، نباید هیچ حافظه‌ای را نشت دهد.

قبل از نوشتن کد بیشتر، مقداری شبیه‌کد بنویسید.

اگر نمی‌دانید چطور مسئله بزرگتر را حل کنید، آن را به مسائل کوچک‌تری تقسیم کنید که می‌توانید آنها را حل کنید. برای مثال این مسئله در واقع چند مسئله است:

1. یک آرگومان خط فرمان را دریافت کنید: اسم کارت حافظه

2. کارت حافظه را باز کنید

3. تا وقتی همچنان داده در حافظه وجود دارد:

a. فایل‌های JPEG از این داده‌ها بسازید

بیایید چند شبکه‌کد برای این مسائل بنویسیم:

```
#include <stdio.h> #include <stdlib.h>

int main(int argc, char *argv[])
{
    // Accept a single command-line argument

    // Open the memory card

    // While there's still data left to read from the memory card

    // Create JPEGs from the data
}
```

شبه‌کدها را به کد تبدیل کنید.

اول به این فکر کنید که چطور یک آرگومان خط فرمان را دریافت کنید. اگر کاربر برنامه را به شکل صحیح استفاده نکرد، باید به او نحوه استفاده صحیح را بگویید.

```
#include <stdio.h> #include <stdlib.h>

int main(int argc, char *argv[])
{
    // Accept a single command-line argument
    if (argc != 2)
    {
        printf("Usage: ./recover FILE\n");
        return 1;
    }

    // Open the memory card

    // While there's still data left to read from the memory card

    // Create JPEGs from the data
}
```

```
#include <stdio.h> #include <stdlib.h>

int main(int argc, char *argv[])
{
    // Accept a single command-line argument
    if (argc != 2)
    {
        printf("Usage: ./recover FILE\n");
        return 1;
    }

    // Open the memory card
    FILE *card = fopen(argv[1], "r");

    // While there's still data left to read from the memory card

    // Create JPEGs from the data
}
```

حال که نحوه استفاده صحیح را بررسی کردید، می‌توانید کارت حافظه را باز کنید. به یاد داشته باشید که می‌توانید `card.raw` را با `fopen` باز کنید.

البته باید بررسی کنید که فایل به درستی باز شده باشد! اگر باز نشد، به کاربر اطلاع دهید و از برنامه خارج شوید. این بخش را به عهده شما می‌گذاریم.

بعد، برنامه شما باید داده‌ها را تا زمانی که هیچ داده‌ای باقی نماند، از کارت حافظه‌ای که باز کردید بخواند. همزمان، برنامه شما باید همه فایل‌های JPEG را از `card.raw` بازیابی کرده و آنها را در فولدر فعلی در یک فایل جدا ذخیره کند

```
#include <stdint.h> #include <stdio.h>

#include <stdlib.h>

int main(int argc, char *argv[])
{
    // Accept a single command-line argument
    if (argc != 2)
    {
        printf("Usage: ./recover FILE\n");
        return 1;
    }

    // Open the memory card
    FILE *card = fopen(argv[1], "r");

    // Create a buffer for a block of data
    uint8_t buffer[512];

    // While there's still data left to read from the memory card
    // Create JPEGs from the data

}
```

اول به این فکر کنید که چطور `card.raw` را بخوانید. به یاد داشته باشید که برای خواندن داده‌های یک فایل، باید آنها را در یک حافظه‌ی موقت “buffer” ذخیره کنید. همچنین به یاد داشته باشید که `card.raw` داده‌ها را در بلوک‌های 512 بایتی ذخیره می‌کند؛ پس بهتر است یک حافظه موقت 512 بایتی بسازید تا همانطور که بلوک‌های داده را می‌خوانید، آنها را ذخیره کنید. یکی از راه‌حل‌های موجود، استفاده از نوع داده `uint8_t` از `stdint.h` است که دقیقاً 8 بیت را ذخیره می‌کند. این نوع داده به این دلیل `uint8_t` نام دارد که عدد صحیح بدون علامت/مثبت/غیر منفی به طول 8 بیت (یک بایت) را ذخیره می‌کند.

ممکن است ایده خوبی نباشد که 512 را به عنوان یک عدد جادویی درون کد خود قرار دهید. شما می‌توانید به شکل بهتری کدنویسی کنید.

حالا به این فکر کنید که چطور داده‌ها را از کارت حافظه بخوانید. با توجه به راهنمای کاربری، `fread` تعداد بایت‌هایی که خوانده است را برمی‌گرداند که در اینجا، با توجه به اینکه `card.raw` حاوی تعدادی بلوک‌های 512 بایتی است، یا 512 و یا 0 را برمی‌گرداند. برای اینکه بعد از باز کردن `card.raw` با `fopen` بتوانیم هر بلوک را بخوانیم، باید از یک حلقه استفاده کنیم:

```
#include <stdint.h> #include <stdio.h>

#include <stdlib.h>

int main(int argc, char *argv[])
{
    // Accept a single command-line argument

    if (argc != 2)
    {
        printf("Usage: ./recover FILE\n");
        return 1;
    }

    // Open the memory card

    FILE *card = fopen(argv[1], "r");
```

```
// Create a buffer for a block of data
uint8_t buffer[512];

// While there's still data left to read from the memory card
while (fread(buffer, 1, 512, card) == 512)
{
    // Create JPEGs from the data
}
}
```

اینگونه، زمانی که `fread` عدد 0، که نشانه‌ی `false` است را بگرداند، حلقه تمام می‌شود. در آخر، این برعهده شماست که بفهمید همزمان با خواندن از `card.raw` چگونه یک فایل JPEG بسازید. برای این کار [walkthrough](#) می‌تواند مفید باشد.

به یاد داشته باشید که برنامه شما باید اسم هر فایل را به شکل `###.jpg` بنویسد و فایل‌ها را شماره‌گذاری کند. `###` یک عدد سه رقمی است که اولین فایل باید 000 باشد. از `sprintf` استفاده کنید و توجه کنید که این تابع یک رشته فرمت شده را در جایی در حافظه ذخیره می‌کند. با توجه به فرمت تعیین‌شده `###.jpg` برای نام فایل‌های JPEG، چند بایت باید برای این رشته اختصاص دهید؟ (کاراکتر NUL را فراموش نکنید!) برای بررسی صحت JPEG هایی که برنامه شما می‌سازد، روی آن دوبار کلیک کنید؛ اگر هر عکس به صورت سالم نمایش داده شود، عملیات شما احتمالاً موفقیت‌آمیز بوده است!

و البته به یاد داشته باشید که هر فایلی که با `fopen` باز کردید را با `fclose` ببندید.

فولدر خود را مرتب نگه دارید.

ممکن است JPEG هایی که اولین نسخه برنامه شما می‌سازد صحیح نباشند. (اگر بعد از باز کردن این فایل‌ها چیزی مشاهده نکردید، احتمالاً آنها درست نیستند) با اجرا کردن دستور زیر تمام JPEG های فولدر خود را حذف کنید.

```
rm *.jpg
```

اگر نمی‌خواهید برای تأیید هر حذف درخواست داده شود، دستور زیر را اجرا کنید.

```
rm -f *.jpg
```

در استفاده از `-f` دقت کنید زیرا حذف‌ها را بدون تأیید شما `force` می‌کند.

نحوه تست

```
./recover card.raw
```

درستی

```
check50 cs50/problems/2025/x/recover
```

استایل

```
style50 recover.c
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/recover
```

CS50x Iran

Harvard's Computer Science 50x Iran

